

Algoritmos I – Prof. Carlos de Salles - Semestre 2026.1 – Primeira Avaliação

Aluno: _____

Cada questão abaixo vale 2,0 pontos e você tem um tempo de 80 minutos de prova.

Assunto da prova: comandos de atribuição, input, print, condicional (if) e funções.

*Bem-vindo a **Neverwinter**, a Jóia do Norte! Após a erupção do Monte Hotenow, a cidade está sendo reconstruída e precisa de aventureiros com habilidades lógicas para organizar seus recursos. Em todas as questões, você deve implementar apenas a função pedida, sem inputs ou a chamada.*

1) O Lorde Dagult Neverember está organizando o estoque de poções de cura para os guardas da cidade. As poções são guardadas em fardos de 6 unidades ou em caixas de 24 unidades. Crie a função **total_pocoas** que recebe a quantidade de **caixas**, **fardos** e **unidades_avulsas** e retorna o número total de poções individuais no inventário.

```
def total_pocoas(caixas, fardos, unidades_avulsas):
```

2) Para entrar na famosa estalagem *Moonstone Mask*, um cliente deve seguir as regras de etiqueta: possuir o convite real ("sim"); **OU** estar vestindo traje de gala ("gala") e ter pelo menos 21 anos; **OU** ser um membro da nobreza ("nobre"), independente de traje ou idade. Crie a função **pode_entrar** que retorna **True** se o acesso for permitido com base nos parâmetros **tem_convite** (string), **traje** (string), **idade** (inteiro) e **eh_nobre** (booleano); ou **False** caso contrário.

```
def pode_entrar(tem_convite, traje, idade, eh_nobre):
```

3) O vulcão próximo à cidade altera drasticamente a temperatura. Os magos precisam monitorar o perigo: se a temperatura for superior a 80 graus, o estado é "crítico"; se estiver entre 50 e 80 (inclusive), é "alerta"; se estiver abaixo de 50, é "normal". No entanto, se o sensor estiver quebrado (sensor_ativo for **False**), a função deve retornar "desconhecido" independente da temperatura. Crie a função **status_vulcão**, que retorna a respectiva string com seu status.

```
def status_vulcão(temperatura, sensor_ativo):
```

4) Para selar um portal para o Plano das Sombras, são necessários quatro cristais mágicos, representados por um inteiro que varia de 1 a 100. Só é possível selar o portal se há dois pares de cristais de mesmo valor. Um desses pares precisa ter dois números pares e o outro par precisa ter dois números ímpares. Crie a função **selar_portal** que retorna **True** caso o portal possa ser selado e **False** em caso contrário.

```
def selar_portal(c1, c2, c3, c4):
```

5) Em um torneio, você dispara três flechas contra os alvos de um rival. O rival fez pontuações fixas de 5, 7 e 9. Você recebe três valores de disparos já ordenados do menor para o maior (menor, meio, maior). Você pode escolher qual flecha sua desafia qual flecha do rival para vencer o maior número de rodadas possível (vence quem tiver o valor estritamente maior). A função **vitórias_torneio** deve retornar o número máximo de vitórias que você consegue obter. Note que $menor \leq meio \leq maior$.

```
def vitórias_torneio(menor, meio, maior):
```

Em Neverwinter, um código limpo é tão valioso quanto uma espada de aço valiriano. Boa sorte, programador!

Uso de IA: foi empregada IA para construção dos textos de cenário.

Questão 1.

Muito bom!

(20)
def total_pacotes(caixas, fardos, unidades_arulhas):

return caixas * 24 + fardos * 6 + unidades_arulhas

Questão 2.

(20)
def pode_entrar(tem_convite, traje, idade, eh_nobre):

if tem_convite == "sim" or (traje == "gala" and idade >= 21) or eh_nobre == True:

return True

return False

Questão 3.

def status_vulcão(temperatura, sensor_ativo):

if sensor_ativo == False:

return "desconhecido" (20)

if temperatura < 50:

return "normal"

elif temperatura <= 80:

return "alerta"

else:

return "crítico"

Questão 4 (próxima folha)

QUESTÃO 4

def par-ou-impar(N1, N2):

if N1 == N2:

if N1 % 2 == 0:

return "par"

else:

return "impar"

return "false"

def selar-paral(C1, C2, C3, C4):

QtdPar, qtdImpar

7.0 Boa estratégia!

if par-ou-impar(C1, C2) == "par":

QtdPar += 1

elif par-ou-impar(C1, C2) == "impar":

qtdImpar += 1

if par-ou-impar(C2, C3) == "par":

qtdPar += 1

elif par-ou-impar(C1, C3) == "impar":

qtdImpar += 1

if par-ou-impar(C1, C4) == "par":

qtdPar += 1

elif par-ou-impar(C1, C4) == "impar":

qtdImpar += 1

if par-ou-impar(C2, C3) == "par":

qtdPar += 1

elif par-ou-impar(C2, C3) == "impar":

qtdImpar += 1

if par-ou-impar(C2, C4) == "par":

qtdPar += 1

elif par-ou-impar(C2, C4) == "impar":

qtdImpar += 1


```
if par-ou-impar(C3,C4) == "par":  
    qtd par += 1  
elif par-ou-impar(C3,C4) == "impar":  
    qtd impar += 1
```

```
if qtd impar == 1 and qtd par == 1:  
    return true  
return false
```

Questão 5:

```
def vitorias_torneio (menor, meio, maior):
```

```
    vitorias
```

```
    if maior > 9:
```

```
        vitorias += 1
```

```
    if meio > 7:
```

```
        vitorias += 1
```

```
    if menor > 5:
```

```
        vitorias += 1
```

```
    if meio > 5:
```

```
        vitorias += 1
```

```
    elif maior > 7:
```

```
        vitorias += 1
```

```
    if meio > 5:
```

```
        vitorias += 1
```

```
    elif maior > 5:
```

```
        vitorias += 1
```

```
    return vitorias
```